Laboratório de Concorrência · Aula 10

Mariana Moreira Lima · RA 24019345-2 · Turma B
Gerado em 15/10/2026, 19:31:44 · tempo registrado no cronômetro: 38:19

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma tarefa depende da outra para continuar. Por exemplo, em um restaurante, o garçom precisa esperar o cozinheiro preparar a comida para entregar ao cliente.

Já a sincronização de competição acontece quando duas pessoas precisam usar o mesmo recurso, mas apenas uma pode utilizar por vez. Por exemplo, duas pessoas querendo usar o mesmo caixa eletrônico precisam esperar sua vez.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que o resultado seja 2.000.000, porque cada thread vai somar 1 ao contador um milhão de vezes.

1ª execução (n = 1.000.000)

1.050.504

2ª execução (n = 1.000.000)

1.094.216

3ª execução (n = 1.000.000)

1.492.199

Saída do programa

```
1ª execução:
esperado: 2000000
obtido: 1050504

2ª execução:
esperado: 2000000
obtido: 1094216

3ª execução:
esperado: 2000000
obtido: 1492199
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, o esperado era 2.000.000, mas os resultados foram 1.050.504, 1.094.216 e 1.492.199. Os valores foram diferentes porque as duas threads acessam e modificam o mesmo contador ao mesmo tempo, fazendo com que algumas atualizações sejam perdidas. Isso é chamado de condição de corrida.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Por exemplo, se o contador estiver em 5, a thread A lê o valor 5 e a thread B também lê 5. As duas somam 1, chegando a 6. Depois, ambas gravam o valor 6 no contador. O correto seria chegar a 7, mas uma atualização foi perdida porque as duas trabalharam com o mesmo valor inicial.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o programa apresentou o resultado esperado de 2000 nas duas execuções. Isso acontece porque, com menos repetições, a condição de corrida pode não aparecer. Esse erro é perigoso porque o programa pode funcionar normalmente durante os testes, mas apresentar resultados incorretos em produção, quando houver mais operações acontecendo ao mesmo tempo.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABBAB → 6

ABBABA → 4

BBAABA → 4

BBBAAA → 7

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis de execução e 4 resultados diferentes: 4, 6, 7 e 8. Apenas uma ordem produz o resultado correto de 8, que é quando a tarefa A termina antes da tarefa B começar.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na sequência AABBBA, a tarefa A lê o valor 3 e soma 1, chegando a 4, mas ainda não grava. Depois, a tarefa B lê o valor 3, multiplica por 2 e grava 6. Por último, a tarefa A grava o valor 4, sobrescrevendo o 6. Assim, a operação de B é perdida e o resultado final fica 4.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão sem sincronização será a mais rápida, porque não precisa controlar o acesso das threads. Porém, ela pode apresentar um resultado incorreto. Já as versões com synchronized e AtomicInteger devem apresentar o resultado correto, mas podem levar mais tempo.

Saída do programa

```
Execução 1:
esperado:          2000000
sem sincronizacao: 1029782 em 2 ms
synchronized:      2000000 em 179 ms
AtomicInteger:     2000000 em 76 ms

Execução 2:
esperado:          2000000
sem sincronizacao: 1031538 em 2 ms
synchronized:      2000000 em 157 ms
AtomicInteger:     2000000 em 25 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com synchronized e AtomicInteger garantem o resultado correto, pois controlam o acesso ao contador compartilhado. Já a versão sem sincronização pode perder atualizações. As versões corretas podem ser mais lentas porque precisam de mecanismos de sincronização para evitar que as threads modifiquem o contador de forma incorreta.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, porque o AtomicInteger protege cada saldo individualmente, mas não garante que as duas operações aconteçam juntas. Se ocorrer um problema entre retirar o dinheiro de uma conta e adicionar na outra, a transferência pode ficar incompleta. Seria necessário utilizar uma transação que garanta que as duas operações sejam realizadas juntas ou que nenhuma seja aplicada.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acredito que as mensagens não vão aparecer, porque a thread A ficará esperando o recurso Y, enquanto a thread B ficará esperando o recurso X. Como cada uma está segurando um recurso que a outra precisa, acontece um impasse e nenhuma consegue terminar.

Saídas (antes e depois da correção)

```
Antes da correção:
IMPASSE: 2 threads presas

Depois da correção:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A estava utilizando o recurso X e esperando o recurso Y. Ao mesmo tempo, a thread B estava utilizando o recurso Y e esperando o recurso X. Como nenhuma liberava o recurso que a outra precisava, aconteceu um deadlock e as duas ficaram bloqueadas.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Quando as threads pegam os recursos sempre na mesma ordem, evitamos que uma fique esperando um recurso que a outra está segurando enquanto espera o primeiro. Assim, uma thread termina e libera os recursos para a outra continuar. Um exemplo seria um sistema bancário em que duas transferências tentam acessar as mesmas contas ao mesmo tempo. Se os recursos forem bloqueados em ordens diferentes, pode acontecer um deadlock.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Acredito que com 3 trabalhadores o programa levará aproximadamente 200 ms, porque as tarefas serão divididas entre eles. Com 1 trabalhador, levará cerca de 600 ms, e com 6 trabalhadores, aproximadamente 100 ms, pois todos poderão executar uma tarefa ao mesmo tempo.

Tempo com 1 trabalhador

levou 600ms

Tempo com 3 trabalhadores

levou 200ms

Tempo com 6 trabalhadores

levou 100ms

Saída do programa

```
3 trabalhador:
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

1 trabalhador:
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

6 trabalhadores:
trabalhador 5 fez a tarefa 6
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
```

```
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as 6 tarefas são executadas uma de cada vez, levando cerca de 600 ms. Com 3 trabalhadores, as tarefas são divididas e levam aproximadamente 200 ms. Com 6 trabalhadores, todas podem ser executadas ao mesmo tempo, levando cerca de 100 ms. O programa não precisa de synchronized porque utiliza canais para distribuir as tarefas, sem que os trabalhadores precisem modificar a mesma variável compartilhada.

Saídas (antes e depois da troca)

```
mensagem de erro:

}
Exit 1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2

depois do ajuste:
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não possui espaço para armazenar mensagens. Quando o programa tenta enviar o valor 1, ele fica esperando alguém receber, mas não existe outra goroutine para isso. Ao criar um canal com espaço para uma mensagem, o valor pode ser armazenado sem precisar esperar. Assim, o programa consegue continuar e mostrar o número 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa vai apresentar um erro de compilação, porque a thread tenta modificar uma variável que pertence à função main. O Rust possui regras de segurança que impedem esse tipo de acesso sem o controle adequado.

Saídas (o erro e o resultado com move)

```
erro:
Exit 1
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
 ⟶ HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                           ^^ may outlive borrowed value `contador`
8 |         contador += 1;                      // a thread altera a variável de main
  |         -------- `contador` is borrowed here
  |
note: function requires argument type to outlive `'static`
 ⟶ HelloWorld.rs:7:13
```

```
   |
 7 |       let t = thread::spawn(|| {
   |  _____________^
 8 | |         contador += 1;                      // a thread altera a variável de main
 9 | |     });
   | |______^
help: to force the closure to take ownership of `contador` (and any other referenced
variables), use the `move` keyword
   |
 7 |     let t = thread::spawn(move || {
   |                           ++++

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
  ⟶ HelloWorld.rs:11:20
    |
 7  |       let t = thread::spawn(|| {
    |               -             -- mutable borrow occurs here
    |  _____________|
    | |
 8  | |         contador += 1;                       // a thread altera a variável de main
    | |         -------- first borrow occurs due to use of `contador` in closure
 9  | |     });
    | |______- argument requires that `contador` is borrowed for `'static`
10  |       t.join().unwrap();
11  |       println!("{}", contador);
    |                      ^^^^^^^^ immutable borrow occurs here
    |
note: requirement that the value outlives `'static` introduced here
  ⟶
/rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.

resultado com move:
0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O programa imprime 0 porque o move faz a thread receber uma cópia do valor da variável contador. Assim, a thread modifica apenas essa cópia, enquanto a variável original da função main continua com o valor 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o problema foi percebido durante a execução, quando o contador apresentou valores diferentes do esperado. Já em Rust, o compilador identificou o problema antes de executar o programa. A vantagem do Rust é evitar erros de compartilhamento de memória, tornando o programa mais seguro. A desvantagem é que o programador precisa seguir regras mais rígidas e utilizar mecanismos como Arc e Mutex.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Acredito que a ordem será A, D, C e B, porque o JavaScript executa primeiro os comandos normais, depois as promessas e por último os temporizadores.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C e B. Isso acontece porque o JavaScript executa primeiro os comandos normais, depois as promessas e por último os temporizadores. Mesmo com tempo 0, o setTimeout precisa esperar sua vez no laço de eventos.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer demorou mais porque o programa ficou ocupado durante 300 ms executando um laço, impedindo que outras tarefas fossem executadas. Em um servidor Node.js, isso pode causar lentidão, pois um cálculo muito pesado pode bloquear o laço de eventos e atrasar o atendimento de outros usuários.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida, em que as duas pessoas conseguem comprar o mesmo lugar porque o sistema verifica a disponibilidade ao mesmo tempo. Para evitar isso, utilizaria uma transação no banco de dados, garantindo que apenas uma compra seja confirmada e que a outra pessoa seja informada de que o ingresso acabou.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi descobrir que duas threads podem executar a mesma operação e produzir resultados diferentes quando não existe sincronização.